

Node.js / Django / Spring Boot / .NET fundamentals

COMPLETE LEARNING GUIDE

What You'll Learn

Almost every backend job today involves working in one of a handful of dominant server frameworks, and each one embodies a different philosophy about how much structure the framework should impose on you. Node.js with Express is minimal and unopinionated — you assemble your own stack. Django is 'batteries included' — ORM, admin panel, auth, and forms ship out of the box. Spring Boot brings convention-over-configuration and a heavyweight dependency-injection container to the JVM, favored in large enterprises. ASP.NET Core is Microsoft's modern, cross-platform rewrite of .NET, spanning full MVC controllers down to terse minimal APIs. This guide compares their request lifecycles, concurrency models, and idiomatic 'hello world' patterns side by side, so you can recognize the same underlying HTTP concepts wearing different syntax, and make an informed choice of framework based on the problem at hand rather than familiarity alone.

1. Four Philosophies, One Job: Turning a Request into a Response

Every one of these frameworks ultimately does the same thing — accept an incoming HTTP request, run it through some handler code, and produce a response — but they disagree sharply on how much of the surrounding machinery they provide for you. Express gives you a router and almost nothing else; you choose your own ORM, validation library, and folder structure. Django gives you an ORM, a templating engine, an admin UI, a auth system, and a strong opinion about project layout, trading flexibility for speed of initial development. Spring Boot sits between the two: it doesn't dictate persistence or templating, but its dependency-injection container and 'starter' dependencies (spring-boot-starter-web, spring-boot-starter-data-jpa) auto-configure huge swaths of an application from a few annotations. ASP.NET Core is the most flexible of the 'batteries included' group, letting you choose between full MVC controllers and lightweight minimal APIs in the same project.

- Express (Node.js): unopinionated, event-driven, single-threaded — you compose middleware and libraries yourself.
- Django (Python): opinionated MVT (Model-View-Template) framework with a built-in ORM, admin panel, and auth system.

- Spring Boot (Java/Kotlin): convention-over-configuration on top of the Spring IoC container; heavy use of annotations and dependency injection.
- ASP.NET Core (C#/.NET): cross-platform (Linux/macOS/Windows) successor to legacy .NET Framework, running on the Kestrel web server.

2. Concurrency Models: How Each Framework Handles Many Requests at Once

The single biggest architectural difference between these frameworks is how they handle concurrent requests, and it directly affects what kind of code is safe to write. Node.js runs on a single-threaded event loop (via libuv): I/O operations like database queries or file reads are non-blocking, so one slow request doesn't block others — but a CPU-heavy synchronous computation freezes the entire server. Django traditionally runs as multiple worker processes (via WSGI servers like gunicorn), each handling one request at a time synchronously, with newer ASGI support enabling async views. Spring Boot, running on the JVM, uses a thread-per-request model by default (a thread pool sized to the servlet container), so a slow synchronous call only blocks that one thread. ASP.NET Core is built around `async/await` from the ground up, using a small pool of threads that are never blocked waiting on I/O, similar in spirit to Node's non-blocking model but multi-threaded.

- Node.js: one thread, non-blocking I/O — great for I/O-bound workloads, dangerous for CPU-bound ones.
- Django/WSGI: multiple worker processes, each synchronous and blocking — scale by adding more workers.
- Spring Boot: thread-per-request on the JVM — a blocking call only ties up one thread out of the pool.
- ASP.NET Core: `async/await` throughout, multi-threaded but non-blocking on I/O — combines both worlds.

3. Minimal 'Hello World' in Express

An Express app starts with almost nothing: import the library, create an app instance, register a route, and start listening. There's no project generator you're required to use, no fixed folder layout, and no built-in database layer — everything beyond routing is a separate npm package you choose and wire in yourself. This minimalism is Express's greatest strength for small services and its biggest liability for large teams, since two Express codebases can end up looking nothing alike.

JAVASCRIPT

```
const express = require("express");
const app = express();

app.get("/", (req, res) => {
  res.status(200).json({ message: "Hello, world!" });
});

app.listen(3000, () => {
  console.log("Server listening on port 3000");
});
```

4. Minimal 'Hello World' in Django

Django separates a request handler ('view') from the URL that routes to it ('URLconf'), and both live inside a Django 'app' — a self-contained module within a larger 'project'. Even a single endpoint touches two files, reflecting Django's assumption that your codebase will eventually be large and needs that separation from day one. JsonResponse is Django's convenience wrapper for returning JSON with the correct Content-Type header set automatically.

PYTHON

```
# views.py
from django.http import JsonResponse

def hello_world(request):
    return JsonResponse({"message": "Hello, world!"})

# urls.py
from django.urls import path
from . import views

urlpatterns = [
    path("hello/", views.hello_world),
]
```

5. Minimal 'Hello World' in Spring Boot

Spring Boot uses annotations to declare intent: @RestController marks a class whose methods return data (not view templates), and @GetMapping binds a method to an HTTP GET route. Behind the scenes, Spring's IoC container scans for these annotations at startup and wires everything together — you never manually instantiate a router or register the class anywhere. This 'convention plus annotation' style is powerful once learned but genuinely opaque to beginners, since a lot happens with no visible call site.

JAVA

```
@RestController
public class HelloController {

    @GetMapping("/hello")
    public ResponseEntity<Map<String, String>> hello() {
        Map<String, String> body = Map.of("message", "Hello, world!");
        return ResponseEntity.ok(body);
    }
}
```

6. Minimal 'Hello World' in ASP.NET Core

Since .NET 6, ASP.NET Core supports 'minimal APIs' — a top-level style much closer to Express than to traditional ASP.NET MVC controllers. `WebApplication.CreateBuilder` sets up the host, dependency injection container, and configuration system in one call; `MapGet` registers a route directly against a lambda, with no controller class required at all. Larger ASP.NET Core apps still typically use full MVC controllers for structure, but minimal APIs are now the recommended starting point for small services and microservices.

C#

```
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

app.MapGet("/hello", () => Results.Ok(new { message = "Hello, world!" }));

app.Run();
```

7. Dependency Injection and Configuration, Framework by Framework

How a framework manages shared objects (database connections, services, configuration) reveals a lot about its design philosophy. Express has no built-in DI container — you pass dependencies around manually or reach for a third-party library. Django uses module-level singletons and `settings.py` as a global configuration object, imported directly wherever needed. Spring's IoC container is the framework's centerpiece: beans are declared once and injected wherever an `@Autowired` constructor parameter asks for them, which is powerful but can obscure where an object actually comes from. ASP.NET Core has first-class built-in dependency injection, registered in `Program.cs` and injected into constructors, striking a middle ground between Spring's heaviness and Express's total absence of a container.

- Express: no built-in DI — dependencies are typically passed explicitly or attached to `req`/`app.locals``.
- Django: global settings module (`settings.py`) plus direct imports of models/services.
- Spring Boot: constructor injection (preferred) or field injection (`@Autowired`) via the IoC container.

- ASP.NET Core: built-in DI container configured in Program.cs, injected via constructor parameters.

8. Choosing a Framework: Practical Decision Criteria

In practice the choice is driven less by raw capability — all four can build the same APIs — and more by team skills, ecosystem fit, and non-functional requirements. Node/Express suits small-to-medium services, real-time features (WebSockets), and teams already fluent in JavaScript across the stack. Django suits content-heavy applications that benefit from a built-in admin panel and ORM, and teams that want fewer early architectural decisions. Spring Boot dominates large enterprise systems, especially where the organization already runs JVM infrastructure and needs mature tooling for transactions, security, and messaging. ASP.NET Core is the natural choice inside a Microsoft-centric stack, or where raw throughput and strong typing matter and the team knows C#. None of these are exclusive — many companies run more than one, using each where it fits best.

Common Pitfalls

- Treating Express like Django — expecting a built-in ORM, admin panel, or auth system that simply isn't there and has to be chosen and wired in yourself.
- Writing a blocking, CPU-heavy synchronous loop in a Node.js request handler, which freezes the single event loop for every concurrent user, not just the one making that request.
- Assuming Django's ORM calls are non-blocking just because the app is deployed under ASGI — most Django ORM code is still synchronous underneath.
- Overusing Spring's field injection (@Autowired on fields) instead of constructor injection, which hides dependencies and makes unit testing much harder.
- Believing '.NET' still means Windows-only — ASP.NET Core has been cross-platform (Linux, macOS, Docker) since .NET Core 1.0.
- Picking a framework based purely on personal familiarity rather than the concurrency model and ecosystem the project actually needs.

Key Takeaways

- All four frameworks map an incoming HTTP request to handler code — the differences are in how much structure and tooling come by default.
- Node's single-threaded event loop is excellent for I/O-bound work but has zero tolerance for blocking, CPU-heavy code in a request handler.
- Django trades flexibility for speed of development by bundling an ORM, admin panel, and auth system out of the box.
- Spring Boot's annotations and IoC container remove boilerplate but require learning to 'read the magic' — prefer constructor injection for testability.

- ASP.NET Core minimal APIs (since .NET 6) let you write Express-style single-file routes without a full MVC controller.
- Framework choice should follow team skills, ecosystem, and concurrency requirements — not just what's most familiar.

Learn More

Express.js — Official Guide

<https://expressjs.com/en/guide/routing.html>

Django Documentation

<https://docs.djangoproject.com/en/stable/>

Spring Boot Reference Documentation

<https://docs.spring.io/spring-boot/documentation.html>